

データベース演習

第8回：データベースを活用するアプリケーション構築

鈴木源吾

前回の復習

1. 集約関数による単純集計

- 集約関数による単純集計
- 例題・演習

2. GROUP BY句の利用

- GROUP BY句で期間ごとに集計
- GROUP BY句でクロス集計？
- 例題・演習

今回のトピック

1. 導入と環境の準備

- 演習に使うPython環境のセットアップ
- セルでのSQL実行：JupySQL

2. Pythonでデータベースアクセス：概要と検索

1. 導入と環境の準備

今回と次回でやること

今回と次回の2回では、Pythonからデータベースにアクセスするプログラムを作成する

- 今回：sqlite3ライブラリを用いて、SQL文を直接実行する基本的な方法を学ぶ
- 次回：データベース操作のフレームワークSQLAlchemyを用いて、より構造的にデータベースを操作する方法を学ぶ

今回は、ノートブックのセルにプログラムを記述し、そのセルを実行して動作を確認するスタイルで進める

使用するPython環境とライブラリ

Google Colab環境でデータベース管理システムSQLiteを使用する。

以下のPythonのライブラリを利用する

- jupy-sql : jupyter notebookのセルでSQLを実行できるライブラリ(JupySQL)
- sqlite : SQLiteデータベースにアクセスするドライバ
- sqlalchemy : データベースにアクセスするフレームワーク（次回使用）

jupy-sqlはインストールが必要である（既の実施経験あり）。

インストールしないと、古いライブラリである、ipython-sqlが使用される

セルでのSQL実行：JupySQL

- JupySQLはjupyter notobookのセルでSQLを実行できるライブラリ
- データベースの中身を簡単に確認できる
- 以下によりライブラリをロードする

```
%load_ext sql
```

- 以下によりSQLiteのデータベースファイルに接続する
 - カレントディレクトリにある, syllabus.sqlite3に接続する
 - 相対パスでも指定できる

```
%sql sqlite:///syllabus.sqlite3
```

セルでのSQL実行：JupySQL

- %%sqlと改行の後にSQL文を書いてセル実行すればよい
- 以下の例は，授業科目のすべての行を出力するSQL文

```
%%sql  
SELECT * FROM 授業科目;
```

- 以下の例は，作成されたテーブルの一覧を出力するSQL文
- sqlite_masterは，SQLiteのメタデータ（システムカタログに相当）の一つ
- テーブルが存在するかどうかを確認するときに使うとよい

```
%%sql  
SELECT * FROM sqlite_master;
```

- 同様にweblogデータベースもSQLで検索してみよう

2. Pythonでデータベースアクセス：概要と検索

ドライバの役割

Pythonによるプログラムからデータベースにアクセスするためには、**ドライバ**と呼ばれるライブラリを使用する必要がある

ドライバの役割

- データベースへのSQL文の送信，結果の取得を行う
 - データベースとの通信機能は，DBMSにインストールされているライブラリを利用する場合と，独自に実装する場合の2通りがある
- データベースにアクセスするためのAPIを提供する
 - API：アプリケーションプログラミングインターフェース
 - データベースにアクセスする関数を使えるようになる

sqlite3パッケージについて

- Pythonから，SQLiteのデータベースにアクセスするドライバ
- Python環境では，デフォルトでインストールされていてすぐ使える
- SQLを使ってデータの検索・作成などができる
- SQLiteはサーバーがなくデータベースがファイルなので，通信機能はない
- sqlite3の公式ドキュメント
<https://docs.python.org/ja/3/library/sqlite3.html>

サンプルプログラム1：データの検索

サンプルプログラム1のセルを実行してみよう

syllabusデータベースの授業科目テーブルのすべてのカラム・行を出力する

```
import sqlite3

connection = sqlite3.connect('syllabus.sqlite3')
cursor = connection.cursor()
cursor.execute('SELECT * FROM 授業科目;') #(2)

for row in cursor :
    print(row) #(3)

cursor.close()
connection.close()
```

サンプルプログラム1：実行結果

サンプルプログラム1のセルを実行すると、以下の実行結果が得られる

```
(1, '確率論', '確率の基礎を学ぶ', '課題, 試験', 1)
(2, 'DB基礎', 'DBの使い方を学ぶ', '試験', 1)
(3, '微分積分', '微分積分の基礎を学ぶ', 'レポート, 試験', 1)
(4, '統計学', '統計的手法を学ぶ', '試験', 2)
(5, 'DB演習', 'DBの応用を学ぶ', 'レポート', 2)
(6, 'IoT演習', 'IoTを体験する', 'レポート', 2)
```

サンプルプログラム1：接続について

```
connection = sqlite3.connect('syllabus.sqlite3')
```

- SQLiteのデータベースファイルを指定してデータベースに接続する
- 接続オブジェクトが生成され、connectionという変数に格納される
- カレントディレクトリにあるsyllabus.sqlite3に接続する
- 相対パスで指定することも可能である
- ユーザ名・パスワードは不要である（PostgreSQLよりもシンプル）

サンプルプログラム1：カーソルとは

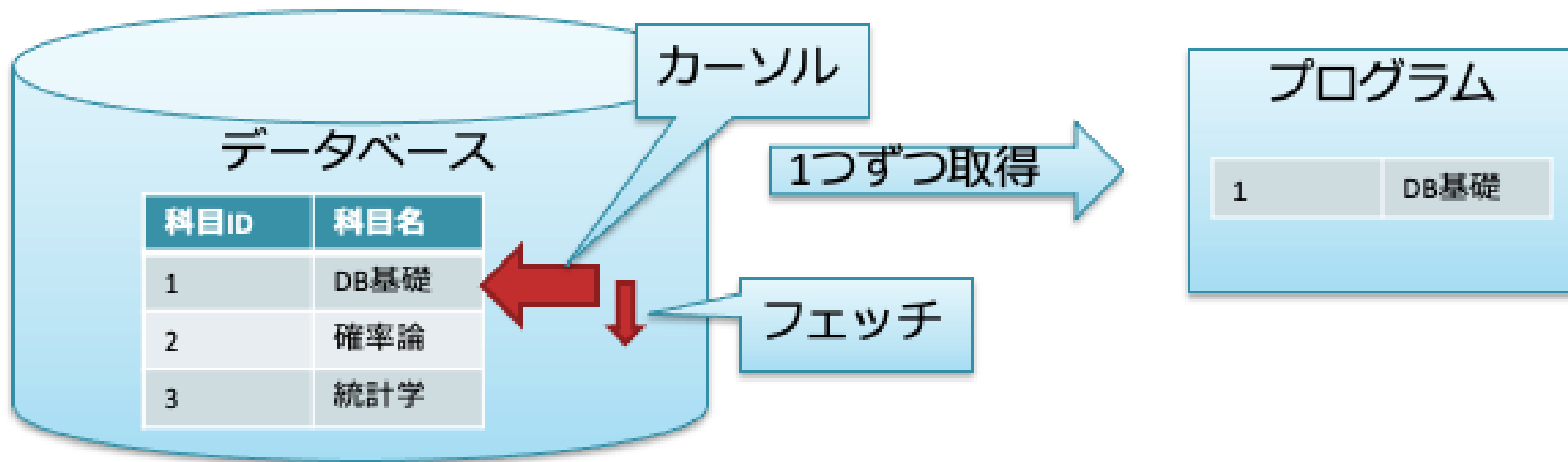
```
cursor = connection.cursor()
```

接続オブジェクトから，cursor関数を用いて，カーソルオブジェクトを生成している
カーソルとは？

- データベースのテーブルには大量に行がある
- 大量の行を一度にプログラムに読み込むとメモリが足りなくなる
- そのため，1行ずつデータを処理する仕組みを実現するのがカーソルである
- カーソルは，検索結果における現在の「位置」を表す概念である
- カーソルを1行ずつ，次の行にずらしていったり，1行ずつ行を取得する
 - 次の行にうつることを，フェッチ（fetch）すると呼ぶ

サンプルプログラム1：カーソルとは

カーソルのイメージを下図に示す。



- しかし、Pythonなどの現代のライブラリでは、プログラミング言語のループの仕組みと非常に融合していて、カーソルをあまり意識する必要がなくなっている
- C言語などでは、もう少し意識して使う必要が出てくる

サンプルプログラム1：カーソルの使い方

```
cursor.execute('select * from 授業科目;')  
  
for row in cursor :  
    print(row)
```

- カーソルオブジェクトに対して、execute関数を実行することで、検索結果の取得が可能になる
- カーソルオブジェクトは、iterableであり（上記のような繰り返し処理が可能という意味）、forループによって、検索結果を取り出すことができる
 - cursorオブジェクトを配列のように扱える
- 上記では、rowに1行の検索結果が入る

終了処理

```
cursor.close()  
connection.close()
```

- 接続オブジェクト・カーソルオブジェクトともに、close関数によって終了処理を行い、接続の解除やメモリの解放を行う必要がある
- 特に、常駐する（常に起動していること）アプリケーションではこのような終了処理は重要であり（終了忘れは、メモリ使用量の増大につながる可能性がある）、普段から忘れない習慣をつけておくこと

検索結果について

検索結果の1行は，以下のように，Pythonのタプルとして得られている

```
(1, '確率論', '確率の基礎を学ぶ', '課題, 試験', 1)
```

Pythonのタプルは，リストによく似ているが，次の違いがある

- タプルは，丸カッコ (()) を使う：(1, 2, 3)
- リストは，角カッコ ([]) を使う：[1, 2, 3]
- タプルは，その要素を変更することができない (immutableと呼ぶ)
- リストは，その要素を変更することができる (mutableと呼ぶ)

要素が1つのタプルは，(1,)とカンマを入れる必要があることに注意すること

- 例えば，サンプルプログラムで，'SELECT 概要 FROM 授業科目;'と修正し，実行してみよう（修正用のセルを用意している）

タプルの要素の取得

- リストと同じ．「[インデックス番号]」を付加してアクセスできる
- 例：a = (1, 2, 3)とすると，a[0]は1，a[1]は2である

サンプルプログラムで，結果の出力である(3)の文を，`print(row)`から，`print(row[1])`に修正すると，以下のような結果が得られる

確率論
DB基礎
微分積分
統計学
DB演習
IoT演習

カーソルの使い方をもう少し

カーソルオブジェクトからデータを取得するのには以下の関数もある

- `fetchone()`：検索結果の次の行を取得し，行を表すタプルを返却する
 - 前の例は，内部的にはこの`fetchone()`が呼ばれている
- `fetchmany()`：検索結果から，引数で指定した個数の行を取得し，結果を行を表すタプルのリストで返却する
- `fetchall()`：検索結果のすべての行を取得し，結果を行を表すタプルのリストで返却する

カーソルの使い方をもう少し (1/2)

カーソルの動きを理解するために、修正用のセルを利用して、検索の後にfetchone()を1回実行し結果を見てみよう。2行を追加している

```
cursor = connection.cursor()
cursor.execute('select * from 授業科目') # (2)
```

```
for row in cursor :
    print(row) # (3)
```

```
tuple = cursor.fetchone()
print(tuple) # (4)
```

この2行を追加

```
cursor.close()
```

カーソルの使い方をもう少し (2/2)

修正したプログラムの実行結果は以下となる．

```
(1, '確率論', '確率の基礎を学ぶ', '課題, 試験', 1)
(2, 'DB基礎', 'DBの使い方を学ぶ', '試験', 1)
(3, '微分積分', '微分積分の基礎を学ぶ', 'レポート, 試験', 1)
(4, '統計学', '統計的手法を学ぶ', '試験', 2)
(5, 'DB演習', 'DBの応用を学ぶ', 'レポート', 2)
(6, 'IoT演習', 'IoTを体験する', 'レポート', 2)
None
```

追加した`print(tuple)`により、`None`が出力されている．つまり、カーソルとは、基本的には順番に上の行から1つずつ動き、最後まで到達するとそこで終わりで、（自前でリストなどに保存しないと）終わった後にもう一度結果を見ることはできない．

検索結果を保持したいときは？

しかし、アプリケーションによっては、検索結果を保持しておき、後で使いたいことがある。そのような場合は、`fetchall()`が便利（行数が少ない場合に限る）

```
cursor.execute('select * from 授業科目') # (2)
result = cursor.fetchall()
print(result)
cursor.close()
```

結果は以下のようにリストになっている（見やすさのために行末を折り返ししているが、リストが1行で表示される）

```
[(1, '確率論', '確率の基礎を学ぶ', '課題, 試験', 1), (2, 'DB基礎', 'DBの使い方を学ぶ', '試験', 1),
 (3, '微分積分', '微分積分の基礎を学ぶ', 'レポート, 試験', 1), (4, '統計学', '統計的手法を学ぶ', '試験', 2),
 (5, 'DB演習', 'DBの応用を学ぶ', 'レポート', 2), (6, 'IoT演習', 'IoTを体験する', 'レポート', 2)]
```


例題1：データの検索

サンプルデータベース（weblog）のcustomersテーブルを用いて、以下のプログラムを作成せよ

- 年齢（カラムcustomer_age）が40歳の顧客について、
- 顧客のID（カラム customer_id）と年齢（カラムcustomer_age）を、
- 5行だけ出力するプログラム（SQL文でlimit 5を使用せよ）.
- 出力は、Pythonのタプルをそのまま出力すること

ノートブックの例題1のセルにプログラムを記述し、セルを実行して動作確認しよう

まずは次ページ以降の解答を見ずに、自力で書いてみよう

例題1：出力例

(88, 40)
(192, 40)
(252, 40)
(288, 40)
(540, 40)

例題1の解答

```
import sqlite3

connection = sqlite3.connect('weblog.sqlite3')
cursor = connection.cursor()

cursor.execute('SELECT customer_id, customer_age FROM customers \
               WHERE customer_age = 40 LIMIT 5;')

for row in cursor :
    print(row)

cursor.close()
connection.close()
```

パラメータを使った検索

値だけを変えて、同じような検索をしたいときがある。

例えば、対象学年が1年生である科目を検索したいとする。発行したいSQL文は以下

```
select * from 授業科目 where 対象学年=1;
```

この1を変数によって変えられるようにしたい。値がyearという変数に入っていたとする。文字列連結を利用する手もあるが (str(year)は数値を文字列に変換)、後述のとおりこの書き方には問題がある

```
year = 1  
sqlstr = 'select * from 授業科目 where 対象学年=' + str(year) + ';;'  
cursor.execute(sqlstr)
```

sqlite3は、より安全で便利なパラメータを使った検索ができる (次ページ)

パラメータを使った検索

sqlite3では、以下のように、SQL文の一部をパラメータ化できる

```
year = 1
cursor.execute('SELECT * FROM 授業科目 WHERE 対象学年 = ? ;', (year,))
```

- ?の部分に、yearの値である1が入る
- "SELECT * FROM 授業科目 WHERE 対象学年=1;"が発行される
- 対象学年が2年生である授業科目を検索したいときは、year=2とするだけで、SQL文は修正する必要がない
- 特に、パラメータは一つの値を何度も使うときに便利である
- パラメータの指定方法（SQLiteは?記号を使用）は、ドライバによって異なる

パラメータを使った検索

2つ以上のパラメータを指定することもできる

```
year = 1  
eval = "試験"  
cursor.execute('SELECT * FROM 授業科目 WHERE 対象学年 = ? AND 評価方法 = ?;', (year, eval))
```

- 最初の?に1が，次の?に'試験'が入る
- "SELECT * FROM 授業科目 WHERE 対象学年 = 1 AND 評価方法 = '試験';"が発行される
- セルの先頭で変数の値（yearやeval）を書き換えてセルを再実行すれば，SQL文を修正せずに様々な条件で検索できる

SQLインジェクション

文字列連結でSQL文を組み立てると、値の中身によっては**意図しないSQL文**になってしまうことがある

例：利用者入力(name)を文字列連結すると…

```
name = "' OR 1=1 --"      # 攻撃者が入れた値
sqlstr = "SELECT * FROM customers WHERE name='" + name + "';"
# 実際のSQL: SELECT * FROM customers WHERE name=' ' OR 1=1 --';
# → WHERE条件が常に真になり、全行が漏えい
```

- 攻撃者が悪意ある値を入れて、データ漏えいや破壊を引き起こす攻撃を **SQLインジェクション** と呼ぶ

パラメータ化で防ぐ

パラメータ化クエリ (?記号) を使えば…

- 値はあくまで「値」として扱われ、SQL構文の一部にならない
 - → SQLインジェクションを防げる
- 型変換やクォート処理もドライバが自動で行う
 - 文字列連結のように `str()` でいちいち変換しなくてよい

→ アプリケーション開発では、パラメータ化を標準的に使うこと

サンプルプログラム2：パラメータを使った検索

以下がサンプルプログラムである

```
import sqlite3

connection = sqlite3.connect('syllabus.sqlite3')
cursor = connection.cursor()
year = 1
eval = "試験"

# 2個パラメータを指定
cursor.execute('SELECT * FROM 授業科目 WHERE 対象学年 = ? AND 評価方法 = ?;', (year, eval))

for row in cursor :
    print(row)

cursor.close()
connection.close()
```

サンプルプログラムの実行結果

サンプルプログラムの実行結果を以下に示す

```
(2, 'DB基礎', 'DBの使い方を学ぶ', '試験', 1)
```

例題2：パラメータを使ったデータの検索

サンプルデータベース（weblog）のcustomersテーブルを用いて、以下のプログラムを作成せよ

- セルの先頭で年齢を変数（例：age）に代入し、
- その年齢（カラムcustomer_age）の顧客を検索し、
- 顧客のID（カラム customer_id）と年齢（カラムcustomer_age）を、
- 5行だけ出力するプログラム（SQL文でlimit 5を使用せよ）。
- 出力は、Pythonのタプルをそのまま出力すること
- パラメータ化（?記号）を使うこと（SQL文へ値を文字列連結しない）
- 変数の値を書き換えて再実行し、異なる年齢でも検索できることを確認すること

まずは次ページ以降の解答を見ずに、自力で書いてみよう

例題2：出力例

年齢を40としたときの出力例

```
(88, 40)  
(192, 40)  
(252, 40)  
(288, 40)  
(540, 40)
```

例題2の解答

```
import sqlite3

age = 40

connection = sqlite3.connect('weblog.sqlite3')
cursor = connection.cursor()

cursor.execute('SELECT customer_id, customer_age FROM customers \
               WHERE customer_age = ? LIMIT 5;', (age,))

for row in cursor :
    print(row)

cursor.close()
connection.close()
```

- ageの値を書き換えてセルを再実行すると、異なる年齢でも検索できる

演習：データの検索

サンプルデータベースのitemsテーブルを用いて、以下のプログラムを作成せよ

- セルの先頭で、ショップのIDと商品の価格を変数（例：shop_id, price）に代入し，
- そのショップIDの店で，その価格より安い商品を検索し，
- その商品のID（カラムitem_id）と価格（カラムitem_price）を，
- 価格の安い順に，
- 最大5行出力するプログラム（SQL文でlimit 5を使用せよ）．
- Pythonのタプルをそのまま出力すること
- パラメータ化（?記号）を使うこと

演習：出力例

ショップIDを1，価格を10000としたときの出力例
(この例だと4行だけしか結果なし)

```
(371, 5910)  
(363, 7530)  
(15, 7820)  
(314, 8640)
```